

四种生成范式的 Tensor Flow 视角：自回归、扩散、流匹配与一致性模型

2026 年 6 月 30 日

目录

1 为什么用 Tensor Flow 视角	3
2 自回归：hidden state 经过 LM head 变成下一个 token	3
2.1 核心张量流	3
2.2 训练时：并行预测所有 next token	3
2.3 推理时：只用最后一个位置	4
2.4 OpenVLA 的 action token 例子	4
2.5 真正的 continuous action head	4
3 扩散：把 clean action/video 张量加噪，再预测噪声	5
3.1 生成对象的 shape	5
3.2 训练时的张量流	5
3.3 推理时的张量流	5
3.4 和 action head 的区别	5
4 流匹配：同样输入中间张量，但预测 velocity	6
4.1 训练样本的构造	6
4.2 模型输入输出	6
4.3 推理时的张量流	6
5 一致性模型：从任意噪声时间点直接映射到 clean sample	7
5.1 模型输入输出	7
5.2 一致性约束	7
5.3 推理	7
6 四种范式的 Head 对比	7
7 以 WAM action chunk 为例的统一对比	7
7.1 自回归 token 版本	8
7.2 连续 action head 版本	8

目录	2
7.3 扩散版本	8
7.4 流匹配版本	8
7.5 一致性版本	9
8 一句话总结	9

1 为什么用 Tensor Flow 视角

这份笔记不从抽象概率论开始，而从模型内部的张量流动开始。我们关心四个问题：

1. 输入张量是什么 shape?
2. backbone 输出什么 latent / hidden state?
3. head 把 latent 解码成什么?
4. loss 在哪些位置、哪些维度上计算?

统一记号如下：

B = batch size, N = token 数或序列长度, D = hidden dimension, V = vocabulary size.

在 LLM/VLA 中常见张量为：

$$H \in \mathbb{R}^{B \times N \times D}.$$

其中 $H[:, i, :]$ 是第 i 个 token 位置的 hidden state。

2 自回归：hidden state 经过 LM head 变成下一个 token

2.1 核心张量流

自回归模型的输入是一串 token：

$$X = [x_1, x_2, \dots, x_N] \in \mathbb{N}^{B \times N}.$$

经过 embedding 和 Transformer：

$$X \xrightarrow{\text{Embed}} E \in \mathbb{R}^{B \times N \times D} \xrightarrow{\text{Transformer}} H \in \mathbb{R}^{B \times N \times D}.$$

LM head 是一个线性层：

$$W_{\text{lm}} \in \mathbb{R}^{D \times V}.$$

因此：

$$\text{logits} = HW_{\text{lm}} \in \mathbb{R}^{B \times N \times V}.$$

这句话非常关键：每一个 token 位置都会输出一个完整词表上的 logits。

2.2 训练时：并行预测所有 next token

训练时使用 teacher forcing。模型输入是：

$$[x_1, x_2, \dots, x_{N-1}],$$

目标是：

$$[x_2, x_3, \dots, x_N].$$

所以第 i 个位置的 hidden state 用来预测第 $i+1$ 个 token：

$$H[:, i, :] \rightarrow \text{logits}[:, i, :] \rightarrow x_{i+1}.$$

交叉熵损失为：

$$\mathcal{L}_{\text{AR}} = \text{CE}(\text{logits}[:, 0 : N-1, :], X[:, 1 : N]).$$

2.3 推理时: 只用最后一个位置

推理时, 当前序列长度为 N 。Transformer 输出:

$$H \in \mathbb{R}^{B \times N \times D}.$$

生成下一个 token 只取最后一个位置:

$$h_{\text{last}} = H[:, -1, :] \in \mathbb{R}^{B \times D}.$$

经过 LM head:

$$\text{logits}_{\text{next}} = h_{\text{last}} W_{\text{lm}} \in \mathbb{R}^{B \times V}.$$

然后从 V 个 token 中选择一个:

$$x_{N+1} \sim \text{Softmax}(\text{logits}_{\text{next}}).$$

2.4 OpenVLA 的 action token 例子

OpenVLA 原版没有单独的 continuous action head。它把连续动作离散化为 action tokens, 然后仍然用 LM head 预测 token。

假设动作是 7 维:

$$a = [a^1, a^2, \dots, a^7].$$

每一维动作先被量化成一个 action token:

$$a^j \rightarrow y_j, \quad y_j \in \{\text{action token ids}\}.$$

推理时不是一次输出 7 维连续向量, 而是自回归生成 7 个 token:

$$y_1, y_2, \dots, y_7.$$

每一步都是:

$$H[:, -1, :] \in \mathbb{R}^{B \times D} \rightarrow \text{LMHead} \rightarrow \mathbb{R}^{B \times V} \rightarrow y_j.$$

最后 ActionTokenizer 把 7 个 action tokens 反量化为连续动作:

$$[y_1, \dots, y_7] \rightarrow [a^1, \dots, a^7].$$

2.5 真正的 continuous action head

如果不用 action token, 而是直接输出连续动作, 则会接一个 action head:

$$h_{\text{last}} \in \mathbb{R}^{B \times D} \xrightarrow{\text{MLP}} \hat{a} \in \mathbb{R}^{B \times A},$$

其中 A 是动作维度, 例如 $A = 7$ 。这时 loss 可以是:

$$\mathcal{L}_{\text{act}} = \|\hat{a} - a\|_1 \quad \text{或} \quad \|\hat{a} - a\|_2^2.$$

3 扩散：把 clean action/video 张量加噪，再预测噪声

3.1 生成对象的 shape

扩散模型处理的是连续张量。比如 WAM 中要生成一个 action chunk：

$$x_0 = a_{t:t+H-1} \in \mathbb{R}^{B \times H \times A}.$$

如果生成未来视频 latent 和动作的联合对象，可以写成：

$$x_0 = [z^o, a] \in \mathbb{R}^{B \times L \times C}.$$

这里 L 是 token/patch/chunk 长度， C 是每个 token 的通道维。

3.2 训练时的张量流

训练时先采样时间步 t ，再采样噪声：

$$\epsilon \sim \mathcal{N}(0, I), \quad \epsilon \in \mathbb{R}^{B \times L \times C}.$$

把 clean sample 加噪：

$$x_t = \alpha_t x_0 + \sigma_t \epsilon.$$

模型输入通常是：

$$(x_t, t, c),$$

其中 c 是条件，例如图像观测、语言指令、proprioception。

denoising network 输出：

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, c) \in \mathbb{R}^{B \times L \times C}.$$

注意：输出 shape 与噪声 shape 完全一样。

loss 是逐元素回归：

$$\mathcal{L}_\epsilon = \|\hat{\epsilon} - \epsilon\|_2^2.$$

3.3 推理时的张量流

推理从纯噪声开始：

$$x_T \sim \mathcal{N}(0, I), \quad x_T \in \mathbb{R}^{B \times L \times C}.$$

每一步：

$$x_t \xrightarrow{\epsilon_\theta(x_t, t, c)} \hat{\epsilon} \xrightarrow{\text{scheduler}} x_{t-1}.$$

重复多步：

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

3.4 和 action head 的区别

扩散模型的输出 head 不是 vocabulary logits，而是一个和 x_t 同 shape 的连续张量：

$$\mathbb{R}^{B \times L \times C} \rightarrow \mathbb{R}^{B \times L \times C}.$$

它不是在词表里选 token，而是在每个连续维度上预测噪声、velocity 或 clean sample。

4 流匹配：同样输入中间张量，但预测 velocity

4.1 训练样本的构造

Flow Matching 也处理连续张量。设：

$$x_0 \sim p_{\text{noise}}, \quad x_1 \sim p_{\text{data}},$$

且二者 shape 相同：

$$x_0, x_1 \in \mathbb{R}^{B \times L \times C}.$$

构造中间状态：

$$x_t = (1 - t)x_0 + tx_1.$$

真实速度为：

$$u_t = x_1 - x_0.$$

4.2 模型输入输出

模型输入：

$$(x_t, t, c).$$

模型输出 velocity：

$$\hat{v} = v_{\theta}(x_t, t, c) \in \mathbb{R}^{B \times L \times C}.$$

loss：

$$\mathcal{L}_{\text{FM}} = \|\hat{v} - (x_1 - x_0)\|_2^2.$$

所以从 tensor flow 上看，Flow Matching 和 diffusion 很像：

input: noisy/intermediate tensor $\rightarrow$ network $\rightarrow$ same-shape continuous tensor.

区别是 target 不同：

$$\text{diffusion target} = \epsilon, \quad \text{flow matching target} = u_t.$$

4.3 推理时的张量流

生成从噪声开始：

$$x(0) \in \mathbb{R}^{B \times L \times C}.$$

每个 ODE step：

$$x(t) \xrightarrow{v_{\theta}(x(t), t, c)} \hat{v}(t) \xrightarrow{\text{ODE solver}} x(t + \Delta t).$$

最终得到：

$$x(1) \approx x_{\text{data}}.$$

5 一致性模型：从任意噪声时间点直接映射到 clean sample

5.1 模型输入输出

Consistency Model 的输入仍是某个时间点的 noisy/intermediate tensor:

$$x_t \in \mathbb{R}^{B \times L \times C}.$$

但模型输出不是噪声，也不是速度，而是 clean sample 的估计：

$$\hat{x}_0 = f_\theta(x_t, t, c) \in \mathbb{R}^{B \times L \times C}.$$

5.2 一致性约束

取同一条轨道上的两个时间点 $s < t$:

$$x_s, x_t.$$

模型应满足：

$$f_\theta(x_t, t, c) \approx f_{\theta^-}(x_s, s, c).$$

loss:

$$\mathcal{L}_{\text{CM}} = d(f_\theta(x_t, t, c), f_{\theta^-}(x_s, s, c)).$$

这里 f_{θ^-} 常是 EMA teacher 或 stop-gradient target。

5.3 推理

一步生成：

$$x_T \sim \mathcal{N}(0, I), \quad \hat{x}_0 = f_\theta(x_T, T, c).$$

少步生成则是：

$$x_T \rightarrow \hat{x}_{t_1} \rightarrow \hat{x}_{t_2} \rightarrow \hat{x}_0.$$

6 四种范式的 Head 对比

范式	backbone 输出	head 输出	输出 shape
自回归	hidden states H	vocab logits	$B \times N \times V$
连续动作 AR	last hidden state	action vector	$B \times A$ 或 $B \times H \times A$
扩散	noisy latent features	noise / velocity / clean tensor	$B \times L \times C$
流匹配	intermediate latent features	velocity tensor	$B \times L \times C$
一致性	noisy/intermediate latent features	clean tensor estimate	$B \times L \times C$

7 以 WAM action chunk 为例的统一对比

假设要生成一段机器人动作：

$$a_{t:t+H-1} \in \mathbb{R}^{B \times H \times A}.$$

7.1 自回归 token 版本

先把每个动作维度离散化：

$$a_{h,j} \rightarrow y_{h,j} \in \{1, \dots, V\}.$$

模型生成 token 序列：

$$y_1, y_2, \dots, y_{H \cdot A}.$$

每一步：

$$H[:, -1, :] \rightarrow \text{LMHead} \rightarrow \mathbb{R}^{B \times V}.$$

最后 detokenize：

$$y_{1:H \cdot A} \rightarrow a_{t:t+H-1}.$$

7.2 连续 action head 版本

Transformer 得到最后 hidden state 或 pooled representation：

$$h \in \mathbb{R}^{B \times D}.$$

action head 直接输出：

$$\hat{a} = \text{MLP}(h) \in \mathbb{R}^{B \times H \times A}.$$

这是并行输出整个 action chunk。

7.3 扩散版本

从 noisy action chunk 开始：

$$a_t^{\text{noisy}} \in \mathbb{R}^{B \times H \times A}.$$

denoising network 每一步输出同 shape 噪声：

$$\hat{\epsilon} \in \mathbb{R}^{B \times H \times A}.$$

多步去噪得到：

$$\hat{a}_{t:t+H-1}.$$

7.4 流匹配版本

从 noise action tensor 到 data action tensor 的中间状态：

$$a_\tau = (1 - \tau)a_0 + \tau a_1.$$

网络输出 velocity：

$$\hat{v} \in \mathbb{R}^{B \times H \times A}.$$

通过 ODE solver 积分得到 action chunk。

7.5 一致性版本

输入任意噪声时间点的 action tensor:

$$a_\tau \in \mathbb{R}^{B \times H \times A}.$$

模型直接输出 clean action chunk:

$$\hat{a} = f_\theta(a_\tau, \tau, c) \in \mathbb{R}^{B \times H \times A}.$$

8 一句话总结

从 tensor flow 视角看, 自回归的核心是

$$B \times N \times D \rightarrow B \times N \times V,$$

即每个 token 位置通过 LM head 预测下一个 token。扩散、流匹配和一致性模型的核心则是

$$B \times L \times C \rightarrow B \times L \times C,$$

区别只在 target: 扩散预测噪声, 流匹配预测速度, 一致性模型预测 clean sample 或轨道终点。